

Mini-Project Vectorized Dynamic BCC-Constrained Centrality (VDBCC)

Muvvala Sairam Suhas (cs23btech11038)
Kasturi Sai Rishi (cs23btech11024)

1 Introduction to the Bottleneck

Closeness Centrality requires computing shortest path distances from every node to all other nodes. This is extremely expensive for large graphs, since it essentially requires running BFS from every node.

The main issue on modern hardware is not just computation, but memory. Standard BFS relies on queue-based traversal, which leads to irregular memory access patterns and heavy cache misses. This is often referred to as the “memory wall”.

There are two major directions people have explored:

- Make BFS faster using hardware (vectorization, parallelism)
- Avoid unnecessary BFS using graph structure (pruning, batching)

Our project combines both.

2 Two Different Definitions of Closeness Centrality

Before combining the two papers, an important detail must be addressed: they use different mathematical definitions.

2.1 Harmonic Closeness (Used in Sariyuce et al.)

$$CC(v) = \sum_{u \neq v} \frac{1}{d(v, u)}$$

This version is easier to compute during BFS because contributions can be added level by level.

2.2 Standard Closeness (Used in Shukla et al.)

$$CC(v) = \frac{n - 1}{\sum_{u \neq v} d(v, u)}$$

This requires computing the total distance sum first and then taking the inverse.

2.3 Our Modification

The vectorized approach naturally supports harmonic centrality, but we want to stay consistent with the dynamic framework of Shukla et al., which uses the standard definition.

A naive implementation would require tracking distance sums separately for each source in the SIMD batch, which breaks vectorization.

We avoid this using the undirected property of the graph:

$$d(u, v) = d(v, u)$$

Instead of each source accumulating distances, each node accumulates distances from all sources using:

$$\text{sumDist}[v] += (\text{number of sources reaching } v) \times \text{level}$$

This allows us to compute standard closeness while still using fast bitwise operations and popcount.

3 The Hardware Approach: Vectorized MS-BFS (Paper 1)

The 2014 paper by Sariyuce et al. removes the memory bottleneck by reformulating BFS as a matrix-style operation.

3.1 Core Idea

Instead of running one BFS at a time, multiple BFS traversals are packed into a single machine word. For example:

- 64-bit integer $\rightarrow$ 64 BFSs
- 256-bit AVX register $\rightarrow$ 256 BFSs

Each bit represents a different source.

Traversal becomes:

- Bitwise OR $\rightarrow$ propagate frontier
- Bitwise AND $\rightarrow$ remove visited nodes

So in one instruction, multiple BFS steps are executed.

3.2 Main Problem

While this reduces memory traffic, it hurts cache locality. The algorithm accesses nodes across the entire graph simultaneously, leading to cache thrashing as the number of concurrent BFSs increases.

4 The Topological Approach: Dynamic Batching (Paper 2)

The 2020 paper by Shukla et al. focuses on avoiding unnecessary computation when the graph changes.

4.1 Core Idea

Instead of recomputing centrality from scratch:

- Process updates in batches
- Identify only affected nodes using distance bounds
- Reduce the graph using structural properties

Key reductions include:

- Chains (degree-2 nodes)
- Redundant structures (R_3, R_4)
- Biconnected Components (BCCs)

The BFS updates are then restricted to the affected BCC.

4.2 Main Problem

Even though the search space is reduced, the BFS itself is still standard and sequential. It does not use SIMD or instruction-level parallelism.

5 Our Proposed Architecture: VDBCC

We combine both ideas: use structural pruning to reduce the problem size, and SIMD vectorization to speed up the computation.

5.1 Key Challenge

Simply combining them does not work:

- SIMD is inefficient for small workloads
- Global vectorization destroys cache locality

We fix this with three design choices.

5.2 BCC-Local Contiguous Memory

Instead of running vectorized BFS on the entire graph, we restrict it to the affected BCC.

We re-index the nodes of the BCC into a contiguous memory block. This improves spatial locality and ensures the data fits into cache.

5.3 Adaptive Threshold Engine

We dynamically choose the execution strategy based on the number of affected nodes:

- Small ($k < 16$): scalar BFS with threads
- Medium ($16 \leq k \leq 64$): 64-bit bitmask
- Large ($k > 64$): AVX-based vectorization

This avoids unnecessary SIMD overhead.

5.4 Delta-Based Bit Propagation

Instead of recomputing full matrix operations, we only propagate bits that changed in the previous level. This reduces redundant work and improves efficiency.

5.5 Standard CC via Symmetry (Our Core Contribution)

We modify the vectorized accumulation step:

- Instead of adding $\frac{1}{l}$ (harmonic)
- We add l multiplied by popcount

$$\text{sumDist}[v] += \text{popcount} \times l$$

This allows us to compute standard closeness centrality without tracking per-source distances.

6 Further improvements on Weighted Networks

The current design assumes unweighted graphs.

To extend to weighted graphs:

- Replace BFS with a parallel SSSP algorithm like Δ -stepping
- Replace equality condition with:

$$d(x, u) + w(u, v) < d(x, v)$$

However, this introduces challenges such as priority queue contention and synchronization overhead. We plan to explore this theoretically as part of the project.